LINUX SECURITY ASSESSMENT – CO5

Answers for Questions 10 to 15

10. Why are rhosts and hosts.equiv considered security risks in Linux? How would you evaluate their impact on system security?

Introduction: The **.rhosts** and **hosts.equiv** files are legacy trust configuration files used by older Linux remote access services such as rlogin and rsh. They allow users from specified remote systems to access another system without repeatedly providing passwords. Although this mechanism was convenient, it creates serious security risks because trust is based mainly on host identity rather than strong modern authentication.

Why they are security risks: If a trusted remote machine is compromised, an attacker may use the existing trust relationship to access other connected systems. Broad trust entries, wildcard configurations, and trusted privileged accounts increase the possibility of unauthorized access and lateral movement.

How to evaluate the impact: The evaluator should check whether these files exist, identify the users and hosts listed in them, and determine whether legacy services such as rsh or rlogin are active. The level of risk becomes high when many systems trust each other or when administrative accounts are included.

Security impact: The overall impact can be severe because compromise of one trusted system may lead to compromise of multiple systems. Therefore, these legacy mechanisms should be disabled and replaced with secure alternatives such as SSH using strong authentication.

11. How would you assess whether an NFS configuration exposes sensitive Linux files to unauthorized users?

Initial assessment: The first step is to inspect the NFS configuration and identify all directories that are exported to remote systems. The evaluator should determine whether any exported directories contain sensitive information such as user data, backups, configuration files, credentials, application secrets, or important system resources.

Access restrictions: The allowed client systems and networks must be reviewed carefully. Allowing access to broad IP ranges or unnecessary hosts increases the chance of unauthorized access. Read-write access should receive greater attention than read-only access because it can allow modification of sensitive data.

Permission verification: Linux file ownership and permissions should also be checked. NFS restrictions alone are not sufficient if the underlying files have weak permissions. Identity mapping and privileged remote access controls should be reviewed to ensure that users cannot improperly gain elevated access.

Risk evaluation: The risk is considered high when sensitive directories are accessible to untrusted systems, writable by unnecessary users, or exposed across large networks. A secure NFS configuration follows the principle of least privilege by exporting only required directories and restricting access to trusted clients.

12. A Linux file has incorrect permissions that allow unauthorized modification. How would you evaluate the resulting security risk?

Identify the importance of the file: The first factor is determining whether the affected file is critical. Modification of a temporary file may have limited consequences, whereas unauthorized modification of system configuration files, executables, scripts, service files, or authentication-related files can seriously affect system security.

Determine who has access: The evaluator should identify which users or groups can modify the file. If every local user has write permission, the risk is much greater than when access is restricted to a small authorized group.

Analyze possible consequences: The way in which the file is used must also be considered. For example, if an attacker can modify a script that is later executed by an administrator or privileged service, the attacker may be able to influence privileged operations.

Evaluate CIA impact: Incorrect write permissions mainly threaten data integrity, but they can also affect confidentiality and availability. Modified files may leak information, introduce malicious behavior, or cause services to fail. Critical files with unauthorized write access should therefore be treated as high-risk security issues and corrected immediately.

13. How would you compare the security impact of weak file permissions with an unpatched Linux vulnerability?

Nature of the weakness: Weak file permissions are configuration-related security problems. They directly give users more access than intended, such as the ability to read, modify, or execute sensitive files. An unpatched vulnerability, on the other hand, is generally a flaw in software or the operating system that may allow an attacker to bypass intended security controls.

Ease of exploitation: Weak permissions can often be exploited easily because an attacker may only need normal access to the system. Unpatched vulnerabilities vary in exploitability; some require specific conditions while others may be remotely accessible and highly dangerous.

Potential impact: Weak permissions can immediately compromise important files and system integrity. An unpatched vulnerability may potentially have a broader impact, especially when it allows remote code execution, privilege escalation, or complete system compromise.

Overall comparison: Both risks should be prioritized based on asset importance, exposure, ease of exploitation, required attacker privileges, and potential damage. Weak permissions are often immediate and predictable risks, while unpatched vulnerabilities require additional analysis of exploitability and available security mitigations. Both should be addressed through permission hardening and regular patch management.

14. How would you evaluate whether a buffer overflow vulnerability in a Linux application is practically exploitable?

Reachability of the vulnerability: The first step is to confirm that the vulnerable code can actually be reached through a realistic input path. In an authorized testing environment, the evaluator should

determine whether specially malformed or excessive input causes abnormal behavior or memory corruption.

Control over program behavior: The assessment should determine whether the memory corruption can influence important program data or control flow. Not every crash automatically means that reliable exploitation is possible, so the actual consequences of the corruption must be analyzed.

Security mitigations: Modern Linux systems include protections such as stack canaries, Address Space Layout Randomization (ASLR), non-executable memory protections, position-independent executables, and compiler hardening. These controls can significantly reduce practical exploitability.

Privilege and exposure: The evaluator should consider whether the application is locally used or remotely accessible, whether authentication is required, and what privileges the vulnerable process has. A remotely reachable flaw in a highly privileged process generally represents greater risk.

Final assessment: Practical exploitability should be judged using reachability, reliability, defensive mitigations, attacker access requirements, privilege impact, and possible consequences. The presence of a buffer overflow alone does not automatically mean complete system compromise.

15. Which factors would you consider before selecting an exploitation technique for a Linux vulnerability?

Authorization and scope: The most important consideration is whether the testing is explicitly authorized and within the approved scope. Security testing should validate the weakness without unnecessarily damaging systems, services, or data.

Type of vulnerability: The chosen approach depends on the nature of the vulnerability. Different categories, such as memory corruption, authentication weaknesses, permission issues, and input-validation flaws, require different methods of assessment.

Target environment: The Linux version, system architecture, installed patches, application version, network exposure, and enabled security controls must be considered. Features such as ASLR, stack protection, mandatory access controls, and containerization may affect the feasibility of testing approaches.

Required privileges and impact: The evaluator should determine what level of access an attacker would need and what privileges could potentially be affected. The potential impact on confidentiality, integrity, and availability should also be assessed.

Reliability and safety: A reliable and non-destructive proof of concept is generally preferable to a disruptive technique. The objective is to demonstrate the actual security risk clearly so that the vulnerability can be properly remediated.

Final decision: Therefore, the selection of an assessment approach should be based on authorization, vulnerability characteristics, target environment, defensive mitigations, reliability, required access, and potential business impact. The main objective should always be accurate risk validation and effective remediation.